Case Study Report

On

Case Study 128 : Credit Card Validation System (C++)

By

Stuti Saxena

150096725024

{Cohort- Larry Page}

1. Introduction to the case study

- Credit cards are widely used for online and offline financial transactions.
- It is important to verify whether the entered card number is valid before processing a transaction.
- Credit card validation systems check the format and mathematical correctness of card numbers.
- This case study develops a **Credit Card Validation System using C++**.
- The system validates card numbers using logical checks and the **Luhn Algorithm**.
    - ❖ The program uses programming concepts such as:
    - ❖ Loops
    - ❖ Conditional statements
    - ❖ Functions
    - ❖ String manipulation
- The system also identifies the **type of card** (Visa, MasterCard, American Express, Discover).
- The project helps students understand real-world applications of programming and validation algorithms.

2. Problem Statement / Case Background

- Many online systems require users to enter credit card numbers for payments.
- Invalid or incorrectly entered card numbers must be detected before processing transactions.
- A validation system ensures that the card number follows standard formatting rules.
- The objective of this case study is to build a Credit Card Validation System in C++.
- The system performs the following tasks:

❖ Accepts a credit card number as input.
❖ Checks if the input contains only digits.
❖ Validates the length of the card number.
❖ Identifies the card provider based on number patterns.
❖ Uses the Luhn Algorithm to check mathematical validity.

- If all checks pass, the system displays “Card Number is Valid” along with the card type.
- If any validation fails, the system displays an appropriate error message.

3. Problem Statement / Case Study Design

- The program is designed as a **menu-driven console application**.
- The user interacts with the system through a simple menu.

Menu Options

- Validate Card
- Exit

Program Flow

1. Program displays the menu.
2. User selects **Validate Card**.
3. User enters the credit card number.
4. The system performs validation steps:
    - Check if all characters are digits.
    - Check card number length.
    - Identify card type.
    - Perform Luhn Algorithm check.
5. The system displays whether the card is **Valid or Invalid**.
6. Program returns to the menu until the user chooses **Exit**.

Functions Used in the Program

- `isAllDigits()` – checks if the card number contains only numeric characters.
- `luhnCheck()` – implements the Luhn Algorithm.
- `getCardType()` – identifies the type of credit card.
- `validateCard()` – performs the complete validation process.
- `main()` – controls the menu system.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

1. Input Validation

- Ensures that the user enters only numeric digits.
- Implemented using the `isdigit()` function.
- Prevents invalid inputs such as letters or special characters.

2. Length Validation

- Credit card numbers have specific lengths.
- The system checks if the card number length is between **13 and 16 digits**.

3. Card Type Identification

The program identifies card types based on starting digits.

Examples:

- **Visa**
  - Starts with 4
  - Length: 13 or 16 digits
- **MasterCard**
  - Starts with 51–55
  - Length: 16 digits
- **American Express**
  - Starts with 34 or 37
  - Length: 15 digits
- **Discover**
  - Starts with 60
  - Length: 16 digits

4. Luhn Algorithm

The Luhn Algorithm checks whether a card number is mathematically valid.

Steps:

- Start from the rightmost digit.
- Double every second digit.
- If the result is greater than 9, subtract 9.
- Add all digits together.
- If the sum is divisible by 10, the card number is valid.

5. Case Study Implementation Details and Snapshots

Programming Language

- C++

Libraries Used

- `<iostream>` for input and output operations.
- `<string>` for handling card numbers as strings.

Implementation Details

- The program uses **functions to divide tasks into smaller parts**.
- Each function performs a specific validation step.
- The system checks:
  - Numeric characters
  - Card number length
  - Card type
  - Luhn algorithm validation

User Interface

- A **menu-driven interface** allows users to select options.
- The program runs inside a **do-while loop** until the user exits.

Screenshots-

Code-

C++CaseStudy

Welcome | creditcard.cpp

creditcard.cpp > luhnCheck(string)

```cpp
#include <iostream>
#include <string>
using namespace std;

// Check if all characters are digits
bool isAllDigits(string card) {
    for (int i = 0; i < card.length(); i++) {
        if (!isdigit(card[i])) {
            return false;
        }
    }
    return true;
}

// Luhn Algorithm - checks if card number is mathematically valid
bool luhnCheck(string card) {
    int sum = 0;
    bool doubleIt = false;

    // Start from the last digit and move left
    for (int i = card.length() - 1; i >= 0; i--) {
        int digit = card[i] - '0';

        if (doubleIt) {
            digit = digit * 2;
            if (digit > 9) {
                digit = digit - 9;
            }
        }

        sum = sum + digit;
        doubleIt = !doubleIt;
    }

    return (sum % 10 == 0);
}
```

Ln 17, Col 17 Spaces: 4 UTF-8 LF {} C++ Go Live Mac

C++CaseStudy

Welcome | creditcard.cpp

creditcard.cpp > luhnCheck(string)

```cpp
bool luhnCheck(string card) {
}

// Identify card type
string getCardType(string card) {
    int len = card.length();
    string firstTwo = card.substr(0, 2);

    if (card[0] == '4' && (len == 13 || len == 16)) {
        return "Visa";
    }
    if ((firstTwo >= "51" && firstTwo <= "55") && len == 16) {
        return "MasterCard";
    }
    if ((firstTwo == "34" || firstTwo == "37") && len == 15) {
        return "American Express";
    }
    if (firstTwo == "60" && len == 16) {
        return "Discover";
    }

    return "Unknown";
}

// Validate and display result
void validateCard() {
    string card;
    cout << "\nEnter Credit Card Number: ";
    cin >> card;

    // Step 1: Check for non-numeric characters
    if (!isAllDigits(card)) {
        cout << "Card Number is Invalid! (Non-numeric characters found)" << endl;
        return;
    }

```

C++CaseStudy

Welcome | creditcard.cpp

creditcard.cpp > luhnCheck(string)

```cpp
void validateCard() {
    // Step 2: Check length
    if (card.length() < 13 || card.length() > 16) {
        cout << "Card Number is Invalid! (Must be 13-16 digits)" << endl;
        return;
    }

    // Step 3: Check card type
    string type = getCardType(card);
    if (type == "Unknown") {
        cout << "Card Number is Invalid! (Unrecognized card format)" << endl;
        return;
    }

    // Step 4: Luhn check
    if (!luhnCheck(card)) {
        cout << "Card Number is Invalid! (Failed Luhn Check)" << endl;
        return;
    }

    // All checks passed
    cout << "Card Number is Valid!" << endl;
    cout << "Card Type  : " << type << endl;
}

int main() {
    int choice;

    do {
        // Menu
        cout << "\n========================" << endl;
        cout << "  Credit Card Validation  " << endl;
        cout << "========================" << endl;
        cout << "1. Validate Card" << endl;
        cout << "2. Exit" << endl;
        cout << "Enter choice: ";
```

Ln 17, Col 17 Spaces: 4 UTF-8 LF {} C++ Go Live Mac

C++CaseStudy

Welcome | creditcard.cpp

creditcard.cpp > luhnCheck(string)

```cpp
int main() {
    do {
        cout << "Enter choice: ";
        cin >> choice;

        if (choice == 1) {
            validateCard();
        } else if (choice == 2) {
            cout << "Goodbye!, thanks for choosing us as your card verifier!" << endl;
        } else {
            cout << "Invalid choice! Enter 1 or 2." << endl;
        }

    } while (choice != 2);

    return 0;
}
```

Output-

C++CaseStudy

EXPLORER

OPEN EDITORS
- Welcome
- creditcard.cpp

C++CASESTUDY
- creditcard
- creditcard.cpp

creditcard.cpp > luhnCheck(string)

```cpp
int main() {
    do {
        cout << "Enter choice: ";
        cin >> choice;

        if (choice == 1) {
            validateCard();
        } else if (choice == 2) {
            cout << "Goodbye!, thanks for choosing us as your card verifier!" << endl;
        } else {
            cout << "Invalid choice! Enter 1 or 2." << endl;
        }

    } while (choice != 2);

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
esktop/  /C++CaseStudy/"creditcard
stutisaxena@Stutis-MacBook-Air-2 C++CaseStudy % cd "/Users/stutisaxena/Desktop/  /C++CaseStudy/" && g++ credi
tcard.cpp -o creditcard && "/Users/stutisaxena/Desktop/  /C++CaseStudy/"creditcard

==========================
  Credit Card Validation
==========================
1. Validate Card
2. Exit
Enter choice: 1

Enter Credit Card Number:
```

Code
Code

OUTLINE
TIMELINE

main* 0 0 Ln 17, Col 17 Spaces: 4 UTF-8 LF C++ Go Live Mac

C++CaseStudy

EXPLORER

OPEN EDITORS
- Welcome
- creditcard.cpp

C++CASESTUDY
- creditcard
- creditcard.cpp

creditcard.cpp > luhnCheck(string)

```cpp
int main() {
    do {
        cout << "Enter choice: ";
        cin >> choice;

        if (choice == 1) {
            validateCard();
        } else if (choice == 2) {
            cout << "Goodbye!, thanks for choosing us as your card verifier!" << endl;
        } else {
            cout << "Invalid choice! Enter 1 or 2." << endl;
        }

    } while (choice != 2);

    return 0;
}
```

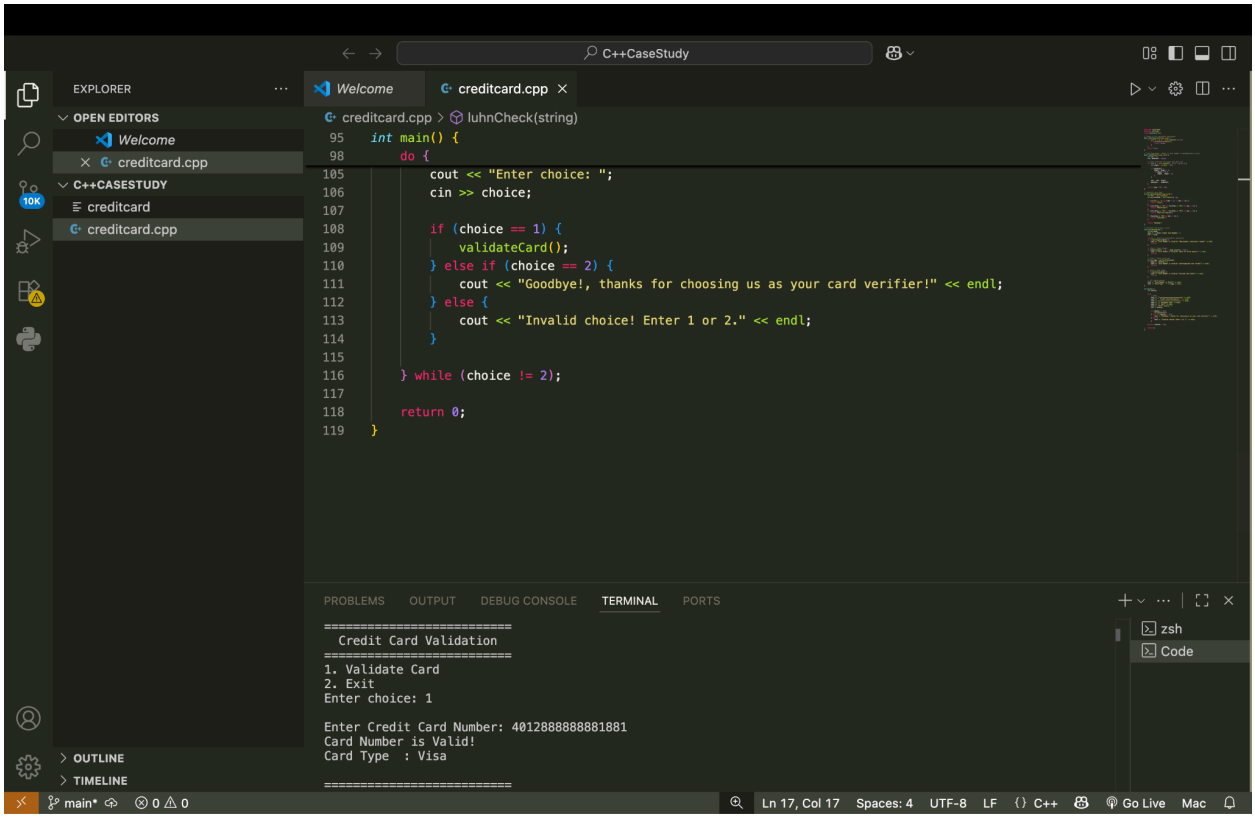
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

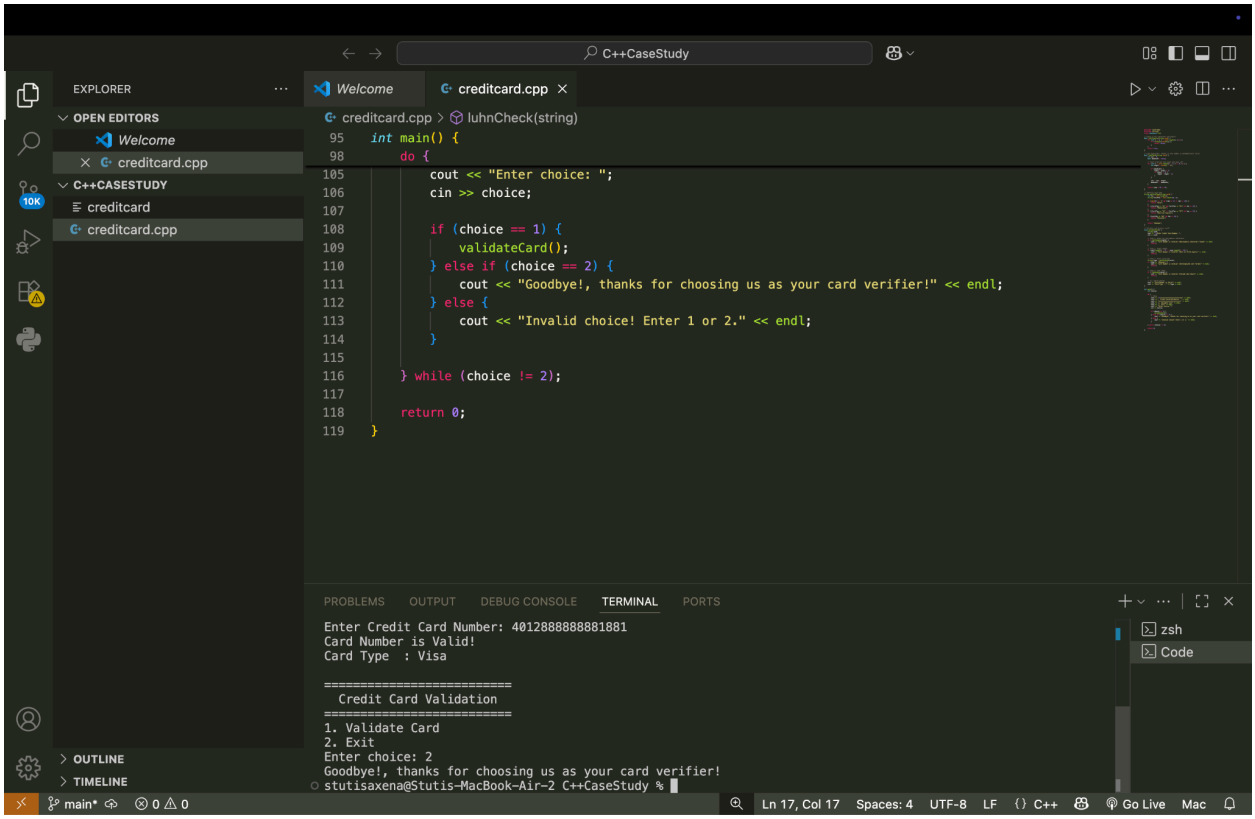
```
cd "/Users/stutisaxena/Desktop/  /C++CaseStudy/" && g++ creditcard.cpp -o creditcard && "/Users/stutisaxena/D
esktop/  /C++CaseStudy/"creditcard
stutisaxena@Stutis-MacBook-Air-2 C++CaseStudy % cd "/Users/stutisaxena/Desktop/  /C++CaseStudy/" && g++ credi
tcard.cpp -o creditcard && "/Users/stutisaxena/Desktop/  /C++CaseStudy/"creditcard

==========================
  Credit Card Validation
==========================
1. Validate Card
2. Exit
Enter choice:
```

Code
Code

OUTLINE
TIMELINE

main* 0 0 Ln 17, Col 17 Spaces: 4 UTF-8 LF C++ Go Live Mac


C++CaseStudy
EXPLORER
OPEN EDITORS
Welcome
creditcard.cpp
C++CASESTUDY
creditcard
creditcard.cpp
OUTLINE
TIMELINE
Welcome
creditcard.cpp
creditcard.cpp > luhnCheck(string)
95 int main() {
98 do {
105 cout << "Enter choice: ";
106 cin >> choice;
108 if (choice == 1) {
109 validateCard();
110 } else if (choice == 2) {
111 cout << "Goodbye!, thanks for choosing us as your card verifier!" << endl;
112 } else {
113 cout << "Invalid choice! Enter 1 or 2." << endl;
114 }
116 } while (choice != 2);
118 return 0;
119 }
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
==========
Credit Card Validation
==========
1. Validate Card
2. Exit
Enter choice: 1
Enter Credit Card Number: 4012888888881881
Card Number is Valid!
Card Type : Visa
==========
zsh
Code
main*
0 0
Ln 17, Col 17
Spaces: 4
UTF-8
LF
C++
Go Live
Mac


C++CaseStudy
EXPLORER
OPEN EDITORS
Welcome
creditcard.cpp
C++CASESTUDY
creditcard
creditcard.cpp
OUTLINE
TIMELINE
Welcome
creditcard.cpp
creditcard.cpp > luhnCheck(string)
95 int main() {
98 do {
105 cout << "Enter choice: ";
106 cin >> choice;
108 if (choice == 1) {
109 validateCard();
110 } else if (choice == 2) {
111 cout << "Goodbye!, thanks for choosing us as your card verifier!" << endl;
112 } else {
113 cout << "Invalid choice! Enter 1 or 2." << endl;
114 }
116 } while (choice != 2);
118 return 0;
119 }
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Enter Credit Card Number: 4012888888881881
Card Number is Valid!
Card Type : Visa
==========
Credit Card Validation
==========
1. Validate Card
2. Exit
Enter choice: 2
Goodbye!, thanks for choosing us as your card verifier!
stutisaxena@Stutis-MacBook-Air-2 C++CaseStudy %
zsh
Code
main*
0 0
Ln 17, Col 17
Spaces: 4
UTF-8
LF
C++
Go Live
Mac

Error Handling-

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

  Credit Card Validation
========================
1. Validate Card
2. Exit
Enter choice: 1

Enter Credit Card Number: 5555555555554440
Card Number is Invalid! (Failed Luhn Check)

========================
  Credit Card Validation
========================
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

  Credit Card Validation
========================
1. Validate Card
2. Exit
Enter choice: 1

Enter Credit Card Number: 1234567890123456
Card Number is Invalid! (Unrecognized card format)

========================
  Credit Card Validation
========================
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

2. Exit
Enter choice: 1

Enter Credit Card Number: 123456789
Card Number is Invalid! (Must be 13-16 digits)

========================
  Credit Card Validation
========================
1. Validate Card
2. Exit
Enter choice:
```

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

2. Exit
Enter choice: 1

Enter Credit Card Number: abcd1234
Card Number is Invalid! (Non-numeric characters found)

========================
  Credit Card Validation
========================
1. Validate Card
2. Exit
Enter choice:
```

6. Problem Statement Results And Conclusion

Results

- The program successfully validates credit card numbers.
- It correctly identifies card types such as:
  - Visa
  - MasterCard
  - American Express
  - Discover
- The system detects invalid inputs such as:
  - Non-numeric characters
  - Incorrect length
  - Invalid card formats
  - Failed Luhn check

Conclusion

- The project demonstrates how **algorithms and programming logic** can be applied to real-world problems.
- It improves understanding of:
  - Input validation
  - Algorithm implementation
  - Structured programming
- The modular design makes the code easy to understand and maintain.

Future Improvements

- Support for more card providers.
- Graphical User Interface (GUI).
- Encryption and security features.
- Database storage for card validation records.

7. References

-C++ Programming Language Documentation
-GeeksforGeeks – Luhn Algorithm
-ISO/IEC 7812 Credit Card Number Standard
-Online C++ Programming Tutorials
-Credit Card Validation Algorithm Resources

SOME TEST NUMBERS TO TEST THE PROJECT-

1. Card numbers that should pass-

A. VISA
   4111111111111111
   4012888888881881

B. MASTER CARD
   5555555555554444
   5105105105105100

C. AMERICAN EXPRESS
   378282246310005
   371449635398431

D. DISCOVER
   6011111111111117
   6011000990139424

2. Card numbers that fail Luhn Check

4111111111111112
6011111111111110
378282246310006
555555555555444

3. Invalid format

1234567890123456
9999999999999999

4.Invalid Length

123456789
41111111111111111

5. Non Numeric

4111abcd11111111
abcd1234